SQL Hands-on Practice (Executable Order)

Author: **Akshatha S**

SQL concepts including **DDL, DML, Aggregate Functions, Subqueries, Joins, and Window Functions**.
This notebook demonstrates SQL concepts including:

- DDL (Database Definition Language)
- DML (Data Manipulation Language)
- Aggregate Functions
- Subqueries
- Joins
- Window Functions

---

1. Database Schema

This project contains the following tables.

Student Table

Column	Data Type
s_id	INT
s_name	VARCHAR(50)
s_age	INT

Employee Table

Column	Data Type	Constraint
emp_id	VARCHAR(20)	PRIMARY KEY
emp_name	VARCHAR(50)	NOT NULL
emp_salary	INT	DEFAULT 20000
emp_dept	VARCHAR(10)	
emp_doj	DATE	NOT NULL
emp_age	INT	CHECK(emp_age>21)

Department Table

Column	Data Type
dept_id	INT
dept_name	VARCHAR(30)
dept_head	VARCHAR(12)

---

2. View Existing Databases

Syntax

```
SHOW DATABASES;
```

Query 1

```
SHOW DATABASES;
```

---

3. Create Database

Syntax

```
CREATE DATABASE database_name;
```

Query 2

```
CREATE DATABASE vtu_university_db;
```

---

4. Verify Database Creation

Query 3

```
SHOW DATABASES;
```

---

5. Select Database

Syntax

```
USE database_name;
```

Query 4

```
USE vtu_university_db;
```

---

6. Create Student Table

Syntax

```
CREATE TABLE table_name(
column datatype
);
```

Query 5

```
CREATE TABLE student(
s_id INT,
s_name VARCHAR(50),
s_age INT
);
```

---

7. View Tables

Query 6

```
SHOW TABLES;
```

---

8. Insert Records into Student Table

Syntax

```
INSERT INTO table_name VALUES(values);
```

Query 7

```
INSERT INTO student VALUES (1,"amit",17);
```

Query 8

```
INSERT INTO student VALUES (2,"navya",25);
```

---

9. Retrieve Student Data

Syntax

```
SELECT * FROM table_name;
```

Query 9

```
SELECT * FROM student;
```

---

10. Insert Duplicate Student Record

Query 10

```
INSERT INTO student VALUES (1,"annapurna",25);
```

---

11. Create Application Database

Query 11

```
CREATE DATABASE app_db;
```

---

12. Use Application Database

Query 12

```
USE app_db;
```

---

13. Create Employee Table with Constraints

Query 13

```
CREATE TABLE employee
(
emp_id VARCHAR(20) PRIMARY KEY,
emp_name VARCHAR(50) NOT NULL,
emp_salary INT DEFAULT 20000,
emp_dept VARCHAR(10),
```

```
emp_doj DATE NOT NULL,
emp_age INT CHECK(emp_age>21)
);
```

---

14. Insert Multiple Employee Records

Query 14

```
INSERT INTO employee VALUES
("ES001","AMIT",50000,"HR","2019-12-03",25),
("ES002","BHARAT",25000,"SALES","2020-01-12",30),
("ES003","CHETAN",40000,"IT","2018-02-12",22),
("ES004","DIYA",60000,"IT","2017-03-02",22),
("ES005","JEEVAN",80000,"HR","2019-12-03",25),
("ES006","PRIYA",75000,"SALES","2020-01-12",30),
("ES007","JOY",90000,"IT","2018-02-12",22),
("ES008","RAVI",45000,"FINANCE","2018-02-03",40),
("ES009","TARUN",65000,"FINANCE","2012-02-03",36);
```

---

15. View Employee Records

Query 15

```
SELECT * FROM employee;
```

---

16. Test Primary Key Constraint

Query 16

```
INSERT INTO employee VALUES
("ES009","ashwin",65000,"FINANCE","2012-02-03",36);
```

---

17. Insert Valid Record

Query 17

```
INSERT INTO employee VALUES
("ES010","ashwin",65000,"FINANCE","2012-02-03",36);
```

---

18. Test Age Constraint

Query 18

```
INSERT INTO employee VALUES
("ES011","drona",65000,"FINANCE","2012-02-03",18);
```

---

19. Insert Values into Specific Columns

Query 19

```
INSERT INTO employee(emp_id,emp_name,emp_doj)
VALUES ("ES012","SHIVANI","2022-12-12");
```

---

20. Test NOT NULL Constraint

Query 20

```
INSERT INTO employee(emp_id,emp_doj)
VALUES ("ES013","2025-12-12");
```

---

21. ALTER TABLE Operations

Query 21

```
ALTER TABLE employee
ADD emp_gender VARCHAR(10);
```

Query 22

```
ALTER TABLE employee
ADD emp_status VARCHAR(15),
ADD emp_state VARCHAR(20);
```

Query 23

```
ALTER TABLE employee
DROP COLUMN emp_status;
```

---

22. Update Records

Query 24

```
UPDATE employee
SET emp_gender="NA"
WHERE emp_gender IS NULL;
```

---

23. Modify Column

Query 25

```
ALTER TABLE employee
MODIFY emp_gender VARCHAR(11) NOT NULL;
```

---

24. Rename Column

Query 26

```
ALTER TABLE employee
CHANGE emp_gender gender VARCHAR(11);
```

---

25. Rename Table

Query 27

```
ALTER TABLE employee
RENAME TO employee_info;
```

---

26. Aggregate Functions

Syntax

```
SELECT aggregate_function(column)
FROM table_name;
```

Query 28 – What is the maximum salary among all employees?

```
SELECT MAX(emp_salary)
FROM employee_info;
```

Query 29 – What is the minimum salary among all employees?

```
SELECT MIN(emp_salary)
FROM employee_info;
```

Query 30 – What is the average salary of employees?

```
SELECT AVG(emp_salary)
FROM employee_info;
```

Query 31 – What is the total salary paid to all employees?

```
SELECT SUM(emp_salary)
FROM employee_info;
```

Query 32 – How many employees are present in the table?

```
SELECT COUNT(emp_id)
FROM employee_info;
```

---

27. GROUP BY Queries

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

Query 33 – How many employees are there in each department?

```
SELECT COUNT(emp_id), emp_dept
FROM employee_info
GROUP BY emp_dept;
```

---

28. DISTINCT Clause

Syntax

```
SELECT DISTINCT column_name
FROM table_name;
```

Query 34 – What are the unique department names present in the employee table?

```
SELECT DISTINCT emp_dept
FROM employee_info;
```

---

29. ORDER BY Clause

Syntax

```
SELECT column_name
FROM table_name
ORDER BY column_name ASC | DESC;
```

Query 35 – How can we sort employee salaries in ascending order?

```
SELECT emp_salary
FROM employee_info
ORDER BY emp_salary ASC;
```

Query 36 – How can we sort employee salaries in descending order?

```
SELECT emp_salary
FROM employee_info
ORDER BY emp_salary DESC;
```

---

30. LIMIT Clause

Syntax

```
SELECT *
FROM table_name
LIMIT number_of_rows;
```

Query 37 – How can we retrieve the first three employee records?

```
SELECT *
FROM employee_info
LIMIT 3;
```

Query 38 – How can we find the employee with the highest salary?

```
SELECT *
FROM employee_info
ORDER BY emp_salary DESC
LIMIT 1;
```

---

31. WHERE Conditions

Syntax

```
SELECT *
FROM table_name
WHERE condition;
```

Query 39 – How can we retrieve employees who belong to the HR department?

```
SELECT *
FROM employee_info
WHERE emp_dept = "HR";
```

Query 40 – How can we find employees whose salary is greater than 50000?

```
SELECT *
FROM employee_info
WHERE emp_salary > 50000;
```

Query 41 – How can we find employees whose salary is between 20000 and 50000?

```
SELECT *
FROM employee_info
WHERE emp_salary BETWEEN 20000 AND 50000;
```

---

32. LIKE Operator

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name LIKE pattern;
```

Query 42 – Which employees have names starting with the letter 'A'?

```
SELECT emp_name
FROM employee_info
WHERE emp_name LIKE "A%";
```

Query 43 – Which employees have names ending with the letter 'A'?

```
SELECT emp_name
FROM employee_info
WHERE emp_name LIKE "%A";
```

---

33. Subqueries

Syntax

```
SELECT *
FROM table_name
WHERE column_name = (subquery);
```

Query 44 – Which employee earns the maximum salary in the company?

```
SELECT *
FROM employee_info
WHERE emp_salary =
(SELECT MAX(emp_salary) FROM employee_info);
```

Query 45 – Which employees earn more than the average salary?

```
SELECT *
FROM employee_info
WHERE emp_salary >
(SELECT AVG(emp_salary) FROM employee_info);
```

Query 46 – Who is the employee with the second highest salary?

```
SELECT *
FROM employee_info
WHERE emp_salary <
(SELECT MAX(emp_salary) FROM employee_info)
ORDER BY emp_salary DESC
LIMIT 1;
```

---

34. Join Database

Query 47 – How can we create a database for practicing join operations?

```
CREATE DATABASE fk_join_db;
```

Query 48 – How can we switch to the join database?

```
USE fk_join_db;
```

---

35. Create Department Table

Query 49 – How can we create a department table?

```
CREATE TABLE department(
dept_id INT PRIMARY KEY,
dept_name VARCHAR(30),
dept_head VARCHAR(12)
);
```

---

36. Create Employee Table with Foreign Key

Query 50 – How can we create an employee table that references the department table?

```
CREATE TABLE employee(
emp_id INT PRIMARY KEY,
emp_name VARCHAR(30),
salary INT,
dept_id INT,
FOREIGN KEY(dept_id) REFERENCES department(dept_id)
);
```

---

37. Insert Department Records

Query 51 – How can we insert department records into the department table?

```
INSERT INTO department VALUES
(10,'HR','Bangalore'),
(20,'IT','Hyderabad'),
(30,'Finance','Mumbai'),
(40,'Sales','Delhi'),
(50,'Marketing','Pune'),
(60,'Admin','Chennai');
```

---

38. Insert Employee Records

Query 52 – How can we insert employee records into the employee table?

```
INSERT INTO employee VALUES
(101,'John',30000,10),
(102,'Mary',45000,20),
(103,'Ravi',50000,20),
(104,'Anita',40000,30),
(105,'Suresh',35000,NULL),
(106,'Neha',28000,10),
(107,'Arjun',60000,20),
(108,'Kiran',32000,40),
(109,'Pooja',38000,50),
(110,'Amit',42000,NULL);
```

---

39. INNER JOIN

Query 53 – How can we retrieve employee names along with their department names?

```
SELECT e.emp_name, d.dept_name
FROM employee e
JOIN department d
ON e.dept_id = d.dept_id;
```

---

40. LEFT JOIN

Query 54 – How can we retrieve all employees even if they are not assigned to any department?

```
SELECT e.emp_name, d.dept_name
FROM employee e
LEFT JOIN department d
ON e.dept_id = d.dept_id;
```

---

41. Employees without Department

Query 55 – Which employees are not assigned to any department?

```
SELECT e.emp_name
FROM employee e
LEFT JOIN department d
ON e.dept_id = d.dept_id
WHERE d.dept_name IS NULL;
```

---

42. Employees in IT Department

Query 56 – Which employees are working in the IT department?

```
SELECT e.emp_name, d.dept_name
FROM employee e
JOIN department d
ON e.dept_id = d.dept_id
WHERE d.dept_name = "IT";
```

---

43. Department Employee Count

Query 57 – How many employees are working in each department?

```
SELECT d.dept_name, COUNT(e.emp_id)
FROM employee e
JOIN department d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

---

44. HAVING Clause

Query 58 – Which departments have more than one employee?

```
SELECT d.dept_name, COUNT(e.emp_id)
FROM employee e
JOIN department d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING COUNT(e.emp_id) > 1;
```

---

45. Window Function Count

Query 59 – How can we count employees in each department using a window function?

```
SELECT *,
COUNT(e.emp_id) OVER(PARTITION BY d.dept_name)
FROM employee e
JOIN department d
ON e.dept_id = d.dept_id;
```

---

46. Window Function Rank

Query 60 – How can we rank employees by salary within each department?

```
SELECT *,
RANK() OVER(PARTITION BY d.dept_name ORDER BY e.salary DESC)
FROM employee e
JOIN department d
ON e.dept_id = d.dept_id;
```

---

47. Overall Salary Ranking

Query 61 – How can we rank all employees based on salary across the entire company?

```
SELECT *,
RANK() OVER(ORDER BY salary DESC)
FROM employee;
```

---

48. Display Employee Records

Query 62 – How can we display all records from the employee table?

```
SELECT * FROM employee;
```

In [ ]: